Air University

Department of Computer Science

Introduction to Parallel & Distributed Computing (CS426L)

Lab Report 07: OpenMP II: Scheduling & Reduction

C# Edition (.NET, Task Parallel Library)

Student Name	Fahad Noor
Registration No.	232448
Section	BSCS-ev-7-C
Instructor	Muhammad Nauman Zafar Hashmi
Date	Monday, October 19, 2026

Section 1: Environment Setup

Confirmed with dotnet --version that the .NET SDK carried over from earlier labs is still available. A working folder for Lab 07 was created containing three console projects: Reduction, SchedulingBench and FalseSharing. Every program was built once with dotnet build -c Release and run with dotnet run --no-build -c Release, passing the number of threads explicitly.

Logical cores (Environment.ProcessorCount): 8

Cache line (bytes): 64 (assumed, the usual size on x86 processors)

```
C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab7_PDC\FalseSharing>dotnet --version
10.0.401
```

Section 2: Task 1 - Parallel Sum and Max: Aggregation vs Locks

Source file: Reduction/Program.cs

```
// Lab 07 - Task 1: Parallel Sum and Max: Aggregation vs Locks
// Five ways to compute the sum and the maximum of a large array:
//   serial       - one thread, the reference result
//   racy         - threads update shared sum/max with no synchronization (WRONG on purpose)
//   lock-each    - threads take a lock for every single element
//   lock-partial - each thread keeps a private sum/max and locks once at the end
//   plinq        - PLINQ Aggregate: thread-local reduction done by the library
//
// Run:  dotnet run --no-build -c Release -- <threads>      e.g.  -- 4

using System;
using System.Diagnostics;
using System.Linq;
using System.Threading;

class Program
{
    const int N = 10_000_000;                // array length
    const int Repeats = 3;                   // each version is timed 3 times and averaged

    static int[] data = Array.Empty<int>();
    static int numThreads = 4;

    // shared results used by racy, lock-each and lock-partial
    static long sharedSum;
    static int sharedMax;
    static readonly object gate = new object();

    // ---------------- serial (reference) ----------------
    static (long sum, int max) SerialVersion()
    {
        long sum = 0;
        int max = int.MinValue;              // identity value for max
        for (int i = 0; i < data.Length; i++)
        {
            sum += data[i];
            if (data[i] > max) max = data[i];
        }
        return (sum, max);
    }

    // Runs body(start, end) on numThreads threads, each on its own block of the array.
    static void RunOnThreads(Action<int, int> body)
    {
        Thread[] threads = new Thread[numThreads];
        int chunk = (data.Length + numThreads - 1) / numThreads;

        for (int t = 0; t < numThreads; t++)
        {
            int start = t * chunk;
            int end = Math.Min(start + chunk, data.Length);
            threads[t] = new Thread(() => body(start, end));
            threads[t].Start();
        }
        foreach (Thread th in threads) th.Join();
```

```
}

// ---------- racy: no synchronization ----------
static (long sum, int max) RacyVersion()
{
    sharedSum = 0;
    sharedMax = int.MinValue;

    RunOnThreads((start, end) =>
    {
        for (int i = start; i < end; i++)
        {
            sharedSum += data[i];                       // unsynchronized read-modify-write
            if (data[i] > sharedMax) sharedMax = data[i];  // unsynchronized check-then-act
        }
    });
    return (sharedSum, sharedMax);
}

// ---------- lock-each: critical section per element ----------
static (long sum, int max) LockEachVersion()
{
    sharedSum = 0;
    sharedMax = int.MinValue;

    RunOnThreads((start, end) =>
    {
        for (int i = start; i < end; i++)
        {
            lock (gate)
            {
                sharedSum += data[i];
                if (data[i] > sharedMax) sharedMax = data[i];
            }
        }
    });
    return (sharedSum, sharedMax);
}

// ---------- lock-partial: private partials, one lock per thread ----------
static (long sum, int max) LockPartialVersion()
{
    sharedSum = 0;
    sharedMax = int.MinValue;

    RunOnThreads((start, end) =>
    {
        long localSum = 0;              // identity for +
        int localMax = int.MinValue;    // identity for max
        for (int i = start; i < end; i++)
        {
            localSum += data[i];
            if (data[i] > localMax) localMax = data[i];
        }

        lock (gate)                     // combine once per thread
        {
            sharedSum += localSum;
            if (localMax > sharedMax) sharedMax = localMax;
        }
    });
    return (sharedSum, sharedMax);
}

// ---------- plinq: library thread-local reduction ----------
static (long sum, int max) PlinqVersion()
{
    return data.AsParallel()
        .WithDegreeOfParallelism(numThreads)
        .Aggregate(
            () => (sum: 0L, max: int.MinValue),                 // seed: each task starts with the identities
            (acc, x) => (acc.sum + x, Math.Max(acc.max, x)),    // fold one element into the task's own partial
            (a, b) => (a.sum + b.sum, Math.Max(a.max, b.max)),  // combine two partials at the end
            result => result);                                  // final result
}

static void Measure(string name, Func<(long sum, int max)> version, (long sum, int max) reference)
{
    double totalMs = 0;
```

```
    (long sum, int max) result = (0, 0);
    bool sumOk = true, maxOk = true;

    for (int r = 0; r < Repeats; r++)
    {
      Stopwatch sw = Stopwatch.StartNew();
      result = version();
      sw.Stop();
      totalMs += sw.Elapsed.TotalMilliseconds;

      if (result.sum != reference.sum) sumOk = false;
      if (result.max != reference.max) maxOk = false;
    }

    Console.WriteLine($"{name,-13} sum={result.sum,-12} max={result.max,-5} " +
                      $"sum_ok={(sumOk ? "YES" : "NO"),-3} max_ok={(maxOk ? "YES" : "NO"),-3} " +
                      $"avg_time={totalMs / Repeats:F2} ms");
  }

  static void Main(string[] args)
  {
    if (args.Length > 0) numThreads = int.Parse(args[0]);

    // fixed seed so every run uses the same data
    Random rng = new Random(12345);
    data = new int[N];
    for (int i = 0; i < N; i++) data[i] = rng.Next(0, 1_000_000);

    (long sum, int max) reference = SerialVersion();

    Console.WriteLine($"Threads: {numThreads}  Elements: {N}  Repeats per version: {Repeats}  ProcessorCount:
{Environment.ProcessorCount}");
    Console.WriteLine($"Reference: sum={reference.sum} max={reference.max}");
    Measure("serial", SerialVersion, reference);
    Measure("racy", RacyVersion, reference);
    Measure("lock-each", LockEachVersion, reference);
    Measure("lock-partial", LockPartialVersion, reference);
    Measure("plinq", PlinqVersion, reference);
  }
}
```

Variants with 4 threads

Variant	Sum correct? (Y/N)	Max correct? (Y/N)	Average time (ms)
serial	Y	Y	6.20
racy	N	Y	32.56
lock-each	Y	Y	255.91
lock-partial	Y	Y	3.72
plinq	Y	Y	17.73

Scaling (average time in ms)

Threads	lock-each	lock-partial	plinq	PLINQ speedup
1	153.96	6.16	30.37	1.00
2	183.43	3.92	19.41	1.56
4	268.75	3.31	13.54	2.24
8	302.40	3.41	13.23	2.30

PLINQ speedup = plinq time with 1 thread / plinq time with n threads.

Observation of the racy version (three runs with 4 threads)

With 4 threads the racy version printed a different wrong sum in every run: 2,546,915,415,651, 3,113,102,803,084 and 2,846,041,044,742 in the three runs, and 2,645,375,662,407 in the 4-thread scaling run, against the correct 4,999,045,030,535. That is 38% to 49% of the total lost. With 2 threads it was 4,359,205,259,894 (12.8% lost), with 8 threads 1,380,002,841,742 (72.4% lost), and with 1 thread it was correct, because one thread cannot race with itself. The maximum (999,999) was correct in every run. The racy version was also about 5 times slower than the serial loop (32.56 ms against 6.20 ms) even though it does no locking.

```
C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab7_PDC\Reduction>dotnet run --no-build -c Release -- 4
Threads: 4  Elements: 10000000  Repeats per version: 3  ProcessorCount: 8
Reference: sum=4999045030535 max=999999
serial        sum=4999045030535 max=999999 sum_ok=YES max_ok=YES avg_time=5.77 ms
racy          sum=2546915415651 max=999999 sum_ok=NO  max_ok=YES avg_time=34.05 ms
lock-each     sum=4999045030535 max=999999 sum_ok=YES max_ok=YES avg_time=265.01 ms
lock-partial  sum=4999045030535 max=999999 sum_ok=YES max_ok=YES avg_time=3.32 ms
plinq         sum=4999045030535 max=999999 sum_ok=YES max_ok=YES avg_time=18.03 ms

C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab7_PDC\Reduction>dotnet run --no-build -c Release -- 4
Threads: 4  Elements: 10000000  Repeats per version: 3  ProcessorCount: 8
Reference: sum=4999045030535 max=999999
serial        sum=4999045030535 max=999999 sum_ok=YES max_ok=YES avg_time=6.53 ms
racy          sum=3113102803084 max=999999 sum_ok=NO  max_ok=YES avg_time=30.98 ms
lock-each     sum=4999045030535 max=999999 sum_ok=YES max_ok=YES avg_time=225.64 ms
lock-partial  sum=4999045030535 max=999999 sum_ok=YES max_ok=YES avg_time=3.52 ms
plinq         sum=4999045030535 max=999999 sum_ok=YES max_ok=YES avg_time=17.75 ms
```

```
C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab7_PDC\Reduction>dotnet run --no-build -c Release -- 4
Threads: 4  Elements: 10000000  Repeats per version: 3  ProcessorCount: 8
Reference: sum=4999045030535 max=999999
serial        sum=4999045030535 max=999999 sum_ok=YES max_ok=YES avg_time=6.29 ms
racy          sum=2846041044742 max=999999 sum_ok=NO  max_ok=YES avg_time=32.66 ms
lock-each     sum=4999045030535 max=999999 sum_ok=YES max_ok=YES avg_time=277.09 ms
lock-partial  sum=4999045030535 max=999999 sum_ok=YES max_ok=YES avg_time=4.33 ms
plinq         sum=4999045030535 max=999999 sum_ok=YES max_ok=YES avg_time=17.41 ms

C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab7_PDC\Reduction>for %t in (1 2 4 8) do dotnet run --no-build -c Release
-- %t

C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab7_PDC\Reduction>dotnet run --no-build -c Release -- 1
Threads: 1  Elements: 10000000  Repeats per version: 3  ProcessorCount: 8
Reference: sum=4999045030535 max=999999
serial        sum=4999045030535 max=999999 sum_ok=YES max_ok=YES avg_time=5.65 ms
racy          sum=4999045030535 max=999999 sum_ok=YES max_ok=YES avg_time=15.30 ms
lock-each     sum=4999045030535 max=999999 sum_ok=YES max_ok=YES avg_time=153.96 ms
lock-partial  sum=4999045030535 max=999999 sum_ok=YES max_ok=YES avg_time=6.16 ms
plinq         sum=4999045030535 max=999999 sum_ok=YES max_ok=YES avg_time=30.37 ms
```

Discussion Questions

1. Why does the racy version give a wrong sum, and was the wrong value the same in every run? Why is the maximum often still correct?

sharedSum += data[i] is a read-modify-write on a shared variable with no synchronization, exactly like counter++ in Lab 04. Two threads read the same old sum, each adds its own element, and the second store overwrites the first, so one element's value is lost. Millions of such collisions happened: with 4 threads 38% to 49% of the total was lost.

The wrong value was different in every run (2,546,915,415,651, 3,113,102,803,084 and 2,846,041,044,742), because it depends on how the threads happen to interleave. More threads lost more: 12.8% with 2 threads and 72.4% with 8; with 1 thread the result was correct.

The maximum stayed correct because max is written very rarely. Once a thread has seen a large value, almost no later element is bigger, so after the first few thousand elements the threads only read sharedMax and never write it. A lost update would also only matter if a smaller value overwrote the true maximum, which needs a write race at exactly the wrong moment. It is still a race and could fail; it is just unlikely, so a correct max is not evidence of correct code.

2. Why is the lock-each version so much slower than the serial loop, and why does it get no faster (or even slower) with more threads?

With 4 threads lock-each took 255.91 ms against 6.20 ms for the serial loop, about 41 times slower. Each of the 10,000,000 elements needs a lock acquire and release, which costs far more than the addition and comparison being protected. The whole loop body is inside the critical section, so the threads cannot do any useful work in parallel: they simply take turns.

Adding threads made it slower: 153.96 ms with 1 thread, 183.43 ms with 2, 268.75 ms with 4 and 302.40 ms with 8. With 1 thread the lock is never contended and only costs its own instructions. With more threads the lock is contended: the lock word and the shared sum bounce between cores' caches, threads spin and sometimes block, and more of every millisecond is spent passing the lock around. Contention grows with the number of threads, while the amount of parallel work stays zero.

3. Explain how the thread-local reduction works: what does each task start with, and when are the partial results combined? What are the identity values for + and for max?

In the PLINQ Aggregate, the seed factory () => (0L, int.MinValue) gives every task its own private accumulator that starts at the identity values. Each task then folds the elements of its own partition into that private accumulator with (acc, x) => (acc.sum + x, Math.Max(acc.max, x)), touching no shared state. Only at the end, after the partitions are finished, does PLINQ combine the partial results pairwise with (a, b) => (a.sum + b.sum, Math.Max(a.max, b.max)) and return the final value.

The identity for + is 0 (x + 0 = x) and the identity for max is the smallest possible value, int.MinValue (max(x, int.MinValue) = x). Starting from the identity means a task that receives no elements does not change the result, and combining any number of partials gives the same answer as the serial loop.

4. Compare lock-partial with plinq. Do they do the same thing, and why is a library aggregation preferable to writing it by hand?

Yes, they compute the same thing in the same way: lock-partial gives each thread a private sum and max starting at the identities, processes one block per thread, and combines the partials under a lock once per thread, i.e. only 4 lock operations in total. PLINQ does the same reduction, but the partitioning, the per-task accumulators and the final combine are done by the library.

On my machine the hand-written version was faster (3.72 ms against 17.73 ms with 4 threads). The work per element is tiny, and PLINQ calls a delegate and builds a tuple for every element, while the hand-written loop is a plain inner loop the JIT can optimize. PLINQ did scale with threads (30.37 ms to 13.23 ms, speedup 2.30 at 8 threads), but it never caught up on such a cheap operation.

A library aggregation is still preferable in most real code: it is shorter, there is no lock or shared variable to get wrong, it chooses the partitioning and load balancing itself, it handles exceptions and cancellation, and it works for any associative operation. Hand-writing the reduction is only worth it in a hot loop where the overhead has been measured, as here.

5. The program uses integers. Why could a floating-point sum computed with a reduction differ slightly between runs or thread counts, even though the program is correct?

Floating-point addition is not associative: (a + b) + c can differ from a + (b + c) in the last bits, because each addition rounds the result to the nearest representable number. A reduction adds the elements in a different grouping from the serial loop: each thread sums its own partition, and then the partial sums are combined. The grouping depends on the number of threads and, with dynamic partitioning such as PLINQ, on which elements each task happened to get, so it can change from run to run.

Each grouping is a correct computation; they are just rounded differently, so results can differ slightly between runs or thread counts even though there is no race. That is why floating-point results of parallel reductions are compared with a small tolerance rather than with ==, and why the lab uses integers, whose addition is exact and associative, so the results can be checked exactly.

Section 3: Task 2 - Benchmarking Static, Dynamic and Guided Scheduling

Source file: SchedulingBench/Program.cs (also used for Task 4 with the uniform argument)

```
// Lab 07 - Task 2 and Task 4: Benchmarking Static, Dynamic and Guided Scheduling
// A loop of N iterations is shared among worker threads using hand-made versions
// of the OpenMP schedules, plus Parallel.For for comparison.
//   uneven workload : iteration i costs work proportional to i (later iterations are heavier)
//   uniform workload: every iteration costs the same
// For every policy the program prints the wall time, the busy time of the busiest and
// least busy worker, the imbalance (busy max / busy min) and whether the result is correct.
// Each policy is run 3 times and the best (fastest) run is reported.
//
// Run:  dotnet run --no-build -c Release -- <threads> [uneven|uniform]
//       e.g.  -- 4           (uneven, Task 2)
//             -- 4 uniform    (uniform, Task 4)

using System;
using System.Collections.Concurrent;
using System.Diagnostics;
using System.Linq;
using System.Threading;
using System.Threading.Tasks;

class Program
{
    const int N = 10_000;                   // loop iterations
    const int CostFactor = 12;              // inner work units per unit of cost
    const int Runs = 3;                     // best of 3

    static int numThreads = 4;
    static bool uniform = false;
    static long[] results = new long[N];

    // ----- the loop body: cost depends on the workload -----
    static long Work(int i)
    {
        int cost = uniform ? CostFactor * N / 2      // same total work as the uneven case
                           : CostFactor * i;         // grows with i
        long acc = 0;
        for (int k = 0; k < cost; k++)
            acc += (k ^ i) & 7;
        return acc;
    }

    // ----- the policies; each worker returns after processing its share -----

    // static: one contiguous block per thread
    static void StaticBlock(int id)
    {
        int chunk = (N + numThreads - 1) / numThreads;
        int start = id * chunk, end = Math.Min(start + chunk, N);
        for (int i = start; i < end; i++) results[i] = Work(i);
    }

    // static,c: chunks of size c dealt out round-robin (thread id, id+T, id+2T, ...)
    static void StaticChunked(int id, int c)
    {
        for (int start = id * c; start < N; start += numThreads * c)
        {
            int end = Math.Min(start + c, N);
            for (int i = start; i < end; i++) results[i] = Work(i);
        }
    }

    // dynamic,c: threads grab the next chunk of size c from a shared counter
    static int nextIteration;
    static void Dynamic(int id, int c)
    {
        while (true)
        {
            int end = Interlocked.Add(ref nextIteration, c);   // claim [end-c, end)
            int start = end - c;
            if (start >= N) break;
            end = Math.Min(end, N);
            for (int i = start; i < end; i++) results[i] = Work(i);
```

```
    }
}

// guided,c: chunk = remaining / threads (never smaller than c), so chunks shrink over time
static void Guided(int id, int c)
{
    while (true)
    {
        int start = Volatile.Read(ref nextIteration);
        if (start >= N) break;

        int remaining = N - start;
        int size = Math.Max(c, (remaining + numThreads - 1) / numThreads);
        int end = Math.Min(start + size, N);

        // claim [start, end) only if nobody else moved the counter in the meantime
        if (Interlocked.CompareExchange(ref nextIteration, end, start) != start)
            continue;                                   // lost the race: try again

        for (int i = start; i < end; i++) results[i] = Work(i);
    }
}

// Runs worker(id) on numThreads threads and records each thread's busy time.
static (double wall, double[] busy) RunThreads(Action<int> worker)
{
    nextIteration = 0;
    double[] busy = new double[numThreads];
    Thread[] threads = new Thread[numThreads];

    Stopwatch wall = Stopwatch.StartNew();
    for (int t = 0; t < numThreads; t++)
    {
        int id = t;                         // private copy for the lambda
        threads[t] = new Thread(() =>
        {
            Stopwatch sw = Stopwatch.StartNew();
            worker(id);
            busy[id] = sw.Elapsed.TotalMilliseconds;
        });
        threads[t].Start();
    }
    foreach (Thread th in threads) th.Join();
    wall.Stop();
    return (wall.Elapsed.TotalMilliseconds, busy);
}

// Parallel.For with the same degree of parallelism; busy time summed per OS thread.
static (double wall, double[] busy) RunParallelFor()
{
    var busyByThread = new ConcurrentDictionary<int, double>();
    var options = new ParallelOptions { MaxDegreeOfParallelism = numThreads };

    Stopwatch wall = Stopwatch.StartNew();
    Parallel.For(0, N, options,
        () => Stopwatch.StartNew(),                     // each task starts its own timer
        (i, state, sw) => { results[i] = Work(i); return sw; },
        sw => busyByThread.AddOrUpdate(Environment.CurrentManagedThreadId,
                                       sw.Elapsed.TotalMilliseconds,
                                       (_, old) => old + sw.Elapsed.TotalMilliseconds));
    wall.Stop();
    return (wall.Elapsed.TotalMilliseconds, busyByThread.Values.ToArray());
}

static void Report(string name, Func<(double wall, double[] busy)> policy, long reference)
{
    double bestWall = double.MaxValue;
    double[] bestBusy = Array.Empty<double>();
    bool correct = true;

    for (int r = 0; r < Runs; r++)
    {
        Array.Clear(results);
        var (wall, busy) = policy();
        if (results.Sum() != reference) correct = false;
        if (wall < bestWall) { bestWall = wall; bestBusy = busy; }
    }
```

```
        double max = bestBusy.Max(), min = bestBusy.Min();
        Console.WriteLine($"{name,-13} wall={bestWall,9:F2} ms  busy_max={max,9:F2} ms  busy_min={min,9:F2} ms  " +
                          $"imbalance={max / min,6:F2}  correct={(correct ? "YES" : "NO")}");
    }

    static void Main(string[] args)
    {
        if (args.Length > 0) numThreads = int.Parse(args[0]);
        if (args.Length > 1) uniform = args[1] == "uniform";

        // serial reference (run once untimed first so the JIT has fully optimized Work)
        long reference = 0;
        for (int i = 0; i < N; i++) reference += Work(i);

        Stopwatch sw = Stopwatch.StartNew();
        long check = 0;
        for (int i = 0; i < N; i++) check += Work(i);
        sw.Stop();

        Console.WriteLine($"Workload: {(uniform ? "uniform" : "uneven")}  Threads: {numThreads}  Iterations: {N}  " +
                          $"ProcessorCount: {Environment.ProcessorCount}  (best of {Runs} runs)");
        Console.WriteLine($"Serial time: {sw.Elapsed.TotalMilliseconds:F2} ms");

        Report("static",       () => RunThreads(id => StaticBlock(id)),       reference);
        Report("static,1",     () => RunThreads(id => StaticChunked(id, 1)),  reference);
        Report("static,64",    () => RunThreads(id => StaticChunked(id, 64)), reference);
        Report("dynamic,1",    () => RunThreads(id => Dynamic(id, 1)),        reference);
        Report("dynamic,64",   () => RunThreads(id => Dynamic(id, 64)),       reference);
        Report("guided,1",     () => RunThreads(id => Guided(id, 1)),         reference);
        Report("guided,64",    () => RunThreads(id => Guided(id, 64)),        reference);
        Report("Parallel.For", RunParallelFor,                                reference);
    }
}
```

Uneven workload with 4 threads (best of 3 runs)

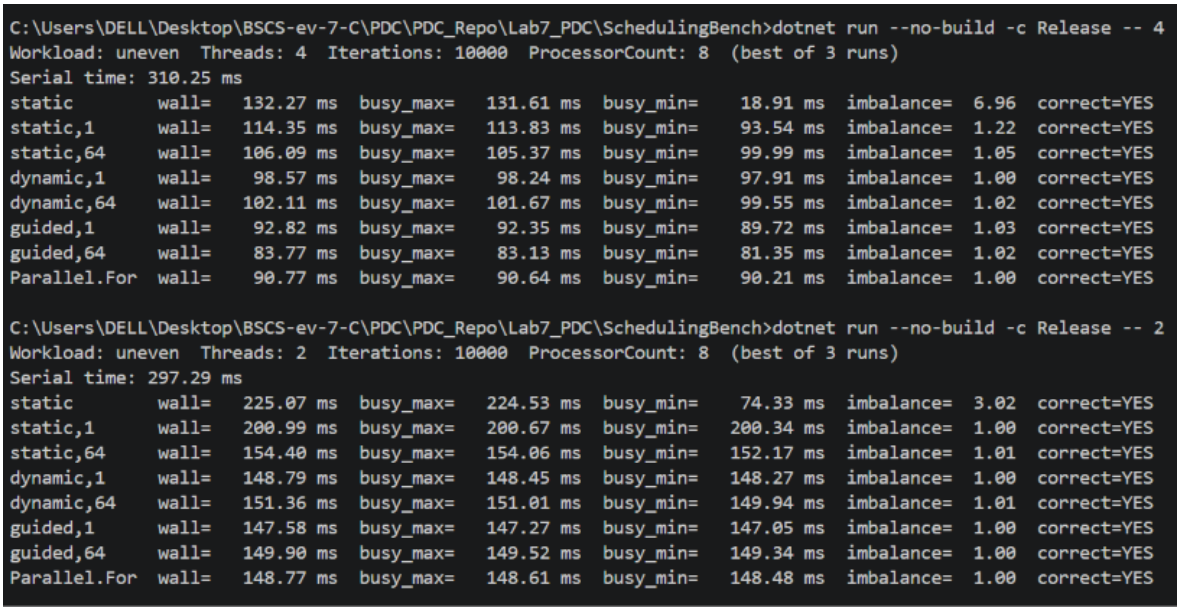
Policy	Wall time (ms)	Busy max (ms)	Busy min (ms)	Imbalance	Correct? (Y/N)
static	132.27	131.61	18.91	6.96	Y
static,1	114.35	113.83	93.54	1.22	Y
static,64	106.09	105.37	99.99	1.05	Y
dynamic,1	98.57	98.24	97.91	1.00	Y
dynamic,64	102.11	101.67	99.55	1.02	Y
guided,1	92.82	92.35	89.72	1.03	Y
guided,64	83.77	83.13	81.35	1.02	Y
Parallel.For	90.77	90.64	90.21	1.00	Y

Imbalance = busy max / busy min (1.00 means perfectly balanced).

Environment.ProcessorCount: 8

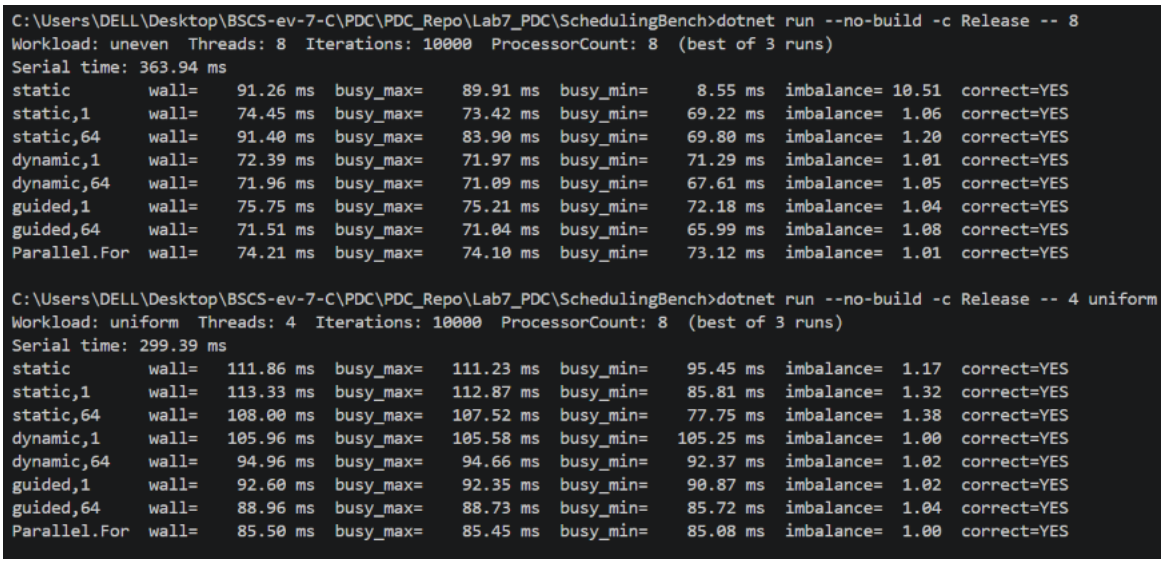
Serial time (ms): 310.25

Fastest policy at 2 / 4 / 8 threads: guided,1 (147.58 ms) / guided,64 (83.77 ms) / guided,64 (71.51 ms)



```
C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab7_PDC\SchedulingBench>dotnet run --no-build -c Release -- 4
Workload: uneven  Threads: 4  Iterations: 10000  ProcessorCount: 8  (best of 3 runs)
Serial time: 310.25 ms
static        wall=   132.27 ms  busy_max=   131.61 ms  busy_min=    18.91 ms  imbalance=  6.96  correct=YES
static,1      wall=   114.35 ms  busy_max=   113.83 ms  busy_min=    93.54 ms  imbalance=  1.22  correct=YES
static,64     wall=   106.09 ms  busy_max=   105.37 ms  busy_min=    99.99 ms  imbalance=  1.05  correct=YES
dynamic,1     wall=    98.57 ms  busy_max=    98.24 ms  busy_min=    97.91 ms  imbalance=  1.00  correct=YES
dynamic,64    wall=   102.11 ms  busy_max=   101.67 ms  busy_min=    99.55 ms  imbalance=  1.02  correct=YES
guided,1      wall=    92.82 ms  busy_max=    92.35 ms  busy_min=    89.72 ms  imbalance=  1.03  correct=YES
guided,64     wall=    83.77 ms  busy_max=    83.13 ms  busy_min=    81.35 ms  imbalance=  1.02  correct=YES
Parallel.For  wall=    90.77 ms  busy_max=    90.64 ms  busy_min=    90.21 ms  imbalance=  1.00  correct=YES

C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab7_PDC\SchedulingBench>dotnet run --no-build -c Release -- 2
Workload: uneven  Threads: 2  Iterations: 10000  ProcessorCount: 8  (best of 3 runs)
Serial time: 297.29 ms
static        wall=   225.07 ms  busy_max=   224.53 ms  busy_min=    74.33 ms  imbalance=  3.02  correct=YES
static,1      wall=   200.99 ms  busy_max=   200.67 ms  busy_min=   200.34 ms  imbalance=  1.00  correct=YES
static,64     wall=   154.40 ms  busy_max=   154.06 ms  busy_min=   152.17 ms  imbalance=  1.01  correct=YES
dynamic,1     wall=   148.79 ms  busy_max=   148.45 ms  busy_min=   148.27 ms  imbalance=  1.00  correct=YES
dynamic,64    wall=   151.36 ms  busy_max=   151.01 ms  busy_min=   149.94 ms  imbalance=  1.01  correct=YES
guided,1      wall=   147.58 ms  busy_max=   147.27 ms  busy_min=   147.05 ms  imbalance=  1.00  correct=YES
guided,64     wall=   149.90 ms  busy_max=   149.52 ms  busy_min=   149.34 ms  imbalance=  1.00  correct=YES
Parallel.For  wall=   148.77 ms  busy_max=   148.61 ms  busy_min=   148.48 ms  imbalance=  1.00  correct=YES
```



```
C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab7_PDC\SchedulingBench>dotnet run --no-build -c Release -- 8
Workload: uneven  Threads: 8  Iterations: 10000  ProcessorCount: 8  (best of 3 runs)
Serial time: 363.94 ms
static        wall=    91.26 ms  busy_max=    89.91 ms  busy_min=     8.55 ms  imbalance= 10.51  correct=YES
static,1      wall=    74.45 ms  busy_max=    73.42 ms  busy_min=    69.22 ms  imbalance=  1.06  correct=YES
static,64     wall=    91.40 ms  busy_max=    83.90 ms  busy_min=    69.80 ms  imbalance=  1.20  correct=YES
dynamic,1     wall=    72.39 ms  busy_max=    71.97 ms  busy_min=    71.29 ms  imbalance=  1.01  correct=YES
dynamic,64    wall=    71.96 ms  busy_max=    71.09 ms  busy_min=    67.61 ms  imbalance=  1.05  correct=YES
guided,1      wall=    75.75 ms  busy_max=    75.21 ms  busy_min=    72.18 ms  imbalance=  1.04  correct=YES
guided,64     wall=    71.51 ms  busy_max=    71.04 ms  busy_min=    65.99 ms  imbalance=  1.08  correct=YES
Parallel.For  wall=    74.21 ms  busy_max=    74.10 ms  busy_min=    73.12 ms  imbalance=  1.01  correct=YES

C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab7_PDC\SchedulingBench>dotnet run --no-build -c Release -- 4 uniform
Workload: uniform  Threads: 4  Iterations: 10000  ProcessorCount: 8  (best of 3 runs)
Serial time: 299.39 ms
static        wall=   111.86 ms  busy_max=   111.23 ms  busy_min=    95.45 ms  imbalance=  1.17  correct=YES
static,1      wall=   113.33 ms  busy_max=   112.87 ms  busy_min=    85.81 ms  imbalance=  1.32  correct=YES
static,64     wall=   108.00 ms  busy_max=   107.52 ms  busy_min=    77.75 ms  imbalance=  1.38  correct=YES
dynamic,1     wall=   105.96 ms  busy_max=   105.58 ms  busy_min=   105.25 ms  imbalance=  1.00  correct=YES
dynamic,64    wall=    94.96 ms  busy_max=    94.66 ms  busy_min=    92.37 ms  imbalance=  1.02  correct=YES
guided,1      wall=    92.60 ms  busy_max=    92.35 ms  busy_min=    90.87 ms  imbalance=  1.02  correct=YES
guided,64     wall=    88.96 ms  busy_max=    88.73 ms  busy_min=    85.72 ms  imbalance=  1.04  correct=YES
Parallel.For  wall=    85.50 ms  busy_max=    85.45 ms  busy_min=    85.08 ms  imbalance=  1.00  correct=YES
```

Discussion Questions

1. Why does the plain static policy show a large imbalance on this workload? Identify which thread (or worker) is busy longest and explain why.

In this workload iteration i costs work proportional to i, so the end of the loop is much more expensive than the start. Plain static gives each of the 4 threads one contiguous quarter. Their shares of the total work are in the ratio 1 : 3 : 5 : 7, so the thread with the last block (iterations 7,500 to 9,999, the last worker) does 7 times the work of the first.

That is exactly what was measured: busy max 131.61 ms, busy min 18.91 ms, imbalance 6.96. The first thread finished after about 19 ms and then sat idle while the last thread kept working, so the wall time (132.27 ms) is set by the last thread. With 2 threads the expected ratio is 3 (measured 3.02) and with 8 threads 15 (measured 10.51).

2. Explain why static,1, the dynamic policies and the guided policies balance the work much better, and what each of them pays for it.

static,1 deals iterations out round-robin, so every thread gets cheap and expensive iterations mixed together and the shares come out nearly equal (imbalance 1.22). It costs nothing at run time, but it has

poor locality, since neighbouring iterations go to different threads, and it still cannot react if a thread runs slower.

The dynamic policies let a thread take the next chunk from a shared counter whenever it finishes the previous one, so faster or luckier threads simply do more chunks and everyone finishes together (imbalance 1.00 for dynamic,1). The price is an atomic operation on a shared counter for every chunk, and that counter's cache line is contended.

Guided does the same with chunks that start large and shrink, so it needs far fewer claims than dynamic,1 but still finishes with small chunks that even out the end (imbalance 1.02 to 1.03). Its claim is a compare-and-swap loop, which is a little more expensive per claim, and the first big chunks can be a risk if they happen to be the expensive ones.

3. How did the results change with 2 and with 8 threads? When the number of threads is larger than the number of cores, what does the busy time of a thread really measure?

With 2 threads the static imbalance was 3.02 and the best policy was guided,1 (147.58 ms), with dynamic,1, Parallel.For and guided,64 within about 2 ms of it. static,1 was noticeably worse here (200.99 ms) even though its imbalance was 1.00. With 8 threads the static imbalance rose to 10.51 and the best policy was guided,64 (71.51 ms), with dynamic,64 at 71.96 ms. The gain from 4 to 8 threads was small (83.77 ms to 71.51 ms), because the 8 logical processors are hyper-threads sharing fewer physical cores.

Busy time is measured with a stopwatch from the moment a worker starts until it finishes, i.e. elapsed wall-clock time, not CPU time. When there are more threads than cores, or threads share a physical core, a thread is sometimes descheduled or running at half speed while its stopwatch keeps running. Its busy time then includes time spent waiting for a core, so it measures how long the thread existed while it had work, not how much computing it actually did.

4. What is the effect of the chunk size? Compare chunk size 1 with chunk size 64 for the dynamic policy, and name the trade-off.

With 4 threads dynamic,1 took 98.57 ms (imbalance 1.00) and dynamic,64 took 102.11 ms (imbalance 1.02); with 8 threads it was 72.39 ms against 71.96 ms. So they were close, and neither size was clearly better here.

The trade-off is overhead against balance. A chunk size of 1 gives the finest balance, because the last chunk any thread takes is a single iteration, but every iteration costs a claim on the shared counter, and the counter's cache line moves between cores constantly. A chunk size of 64 makes 64 times fewer claims and gives better locality, but at the end a thread can be left holding up to 64 iterations while the others are idle. In this workload each iteration takes tens of microseconds, so 10,000 claims are cheap and the two sizes perform similarly; the claim overhead would dominate if the iterations were very cheap, and large chunks would hurt if the iterations were very expensive.

5. Why does guided start with large chunks and then shrink them? In what kind of workload would that be an advantage over dynamic,1?

Guided sets the chunk size to roughly the remaining work divided by the number of threads. At the start there is a lot of work left, so the chunks are large and only a few claims are needed. As the loop runs down, the chunks shrink, so near the end the work is split into small pieces and all threads finish at almost the same time. It aims to get the low overhead of large chunks and the good final balance of small ones.

It is an advantage over dynamic,1 when the iterations are cheap enough that a claim per iteration is a noticeable cost, or when there are very many iterations, and the cost does not jump up near the start. It also suits loops like this one where the early iterations are cheap. It is weaker when the expensive iterations come first, because one thread can then be stuck with a large, expensive first chunk.

Section 4: Task 3 - False Sharing and Padding

Source file: FalseSharing/Program.cs

```
// Lab 07 - Task 3: False Sharing and Padding
// Every thread increments ONLY its own counter, so the program is always correct.
//   adjacent          - counters[t]: neighbouring counters share one 64-byte cache line
//   padded            - counters[t * Padding]: each counter on its own cache line
//   local-accumulate  - count in a local variable, write the counter once at the end
//
// Run:  dotnet run --no-build -c Release -- <threads>      e.g.  -- 4

using System;
using System.Diagnostics;
using System.Threading;

class Program
{
    const int IncrementsPerThread = 100_000_000;
    const int CacheLineBytes = 64;
    const int Padding = 16;                  // 16 longs = 128 bytes between counters (> one cache line)

    static int numThreads = 4;

    static double RunThreads(Action<int> body)
    {
        Thread[] threads = new Thread[numThreads];
        Stopwatch sw = Stopwatch.StartNew();
        for (int t = 0; t < numThreads; t++)
        {
            int id = t;                      // private copy for the lambda
            threads[t] = new Thread(() => body(id));
            threads[t].Start();
        }
        foreach (Thread th in threads) th.Join();
        sw.Stop();
        return sw.Elapsed.TotalMilliseconds;
    }

    static void Main(string[] args)
    {
        if (args.Length > 0) numThreads = int.Parse(args[0]);

        // ---------- adjacent: 8-byte counters packed next to each other ----------
        long[] adjacent = new long[numThreads];
        double adjacentMs = RunThreads(id =>
        {
            for (int i = 0; i < IncrementsPerThread; i++)
                adjacent[id]++;              // writes to memory every iteration
        });

        // ---------- padded: each counter 128 bytes away from the next ----------
        long[] padded = new long[(numThreads + 1) * Padding];
        double paddedMs = RunThreads(id =>
        {
            int slot = (id + 1) * Padding;     // +1 also keeps slot 0 away from the array header
            for (int i = 0; i < IncrementsPerThread; i++)
                padded[slot]++;
        });

        // ---------- local-accumulate: no shared writes inside the loop ----------
        long[] results = new long[numThreads];
        double localMs = RunThreads(id =>
        {
            long local = 0;
            for (int i = 0; i < IncrementsPerThread; i++)
                local++;
            results[id] = local;             // one write per thread
        });

        bool correct = true;
        for (int t = 0; t < numThreads; t++)
        {
            if (adjacent[t] != IncrementsPerThread) correct = false;
            if (padded[(t + 1) * Padding] != IncrementsPerThread) correct = false;
            if (results[t] != IncrementsPerThread) correct = false;
        }
```

```
    Console.WriteLine($"Threads: {numThreads}  Increments per thread: {IncrementsPerThread}  " +
                      $"Cache line (assumed): {CacheLineBytes} bytes  ProcessorCount: {Environment.ProcessorCount}");
    Console.WriteLine($"adjacent          time={adjacentMs:F2} ms");
    Console.WriteLine($"padded            time={paddedMs:F2} ms");
    Console.WriteLine($"local-accumulate  time={localMs:F2} ms");
    Console.WriteLine($"Slowdown adjacent/padded: {adjacentMs / paddedMs:F2}x");
    Console.WriteLine($"Counters correct={(correct ? "YES" : "NO")}");
  }
}
```

Three runs with 4 threads

Run	adjacent (ms)	padded (ms)	local-accumulate (ms)	Counters correct? (Y/N)
1	549.11	134.43	40.11	Y
2	620.19	130.84	40.58	Y
3	548.19	132.36	35.13	Y

Effect of the number of threads

Threads	adjacent (ms)	padded (ms)	Slowdown adjacent / padded
1	130.06	128.84	1.01x
2	285.51	129.31	2.21x
4	572.50 (avg of 3 runs)	132.54 (avg of 3 runs)	4.32x
8	837.82	141.15	5.94x

Cache-line size (bytes): 64 (assumed, the usual size on x86 processors)

Environment.ProcessorCount: 8

```
C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab7_PDC\FalseSharing>dotnet run --no-build -c Release -- 4
Threads: 4  Increments per thread: 100000000  Cache line (assumed): 64 bytes  ProcessorCount: 8
adjacent          time=549.11 ms
padded            time=134.43 ms
local-accumulate  time=40.11 ms
Slowdown adjacent/padded: 4.08x
Counters correct=YES

C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab7_PDC\FalseSharing>dotnet run --no-build -c Release -- 4
Threads: 4  Increments per thread: 100000000  Cache line (assumed): 64 bytes  ProcessorCount: 8
adjacent          time=620.19 ms
padded            time=130.84 ms
local-accumulate  time=40.58 ms
Slowdown adjacent/padded: 4.74x
Counters correct=YES
```

```
C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab7_PDC\FalseSharing>dotnet run --no-build -c Release -- 4
Threads: 4  Increments per thread: 100000000  Cache line (assumed): 64 bytes  ProcessorCount: 8
adjacent          time=548.19 ms
padded            time=132.36 ms
local-accumulate  time=35.13 ms
Slowdown adjacent/padded: 4.14x
Counters correct=YES

C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab7_PDC\FalseSharing>for %t in (1 2 8) do dotnet run --no-build -c Release
 -- %t

C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab7_PDC\FalseSharing>dotnet run --no-build -c Release -- 1
Threads: 1  Increments per thread: 100000000  Cache line (assumed): 64 bytes  ProcessorCount: 8
adjacent          time=130.06 ms
padded            time=128.84 ms
local-accumulate  time=24.49 ms
Slowdown adjacent/padded: 1.01x
Counters correct=YES
```

Discussion Questions

1. Why is the program with adjacent counters correct but slower, although no two threads ever write to the same variable? Explain in terms of cache lines and invalidation.

Every thread increments only its own element adjacent[id], so no two threads ever write the same variable and the result is always correct (counters correct=YES in every run). The problem is the cache line. The processor moves memory between cores in 64-byte lines, and adjacent long counters are 8 bytes apart, so all the counters sit in the same line.

Before a core can write to a line it must own it exclusively, which invalidates the copies in every other core. Thread 0 writes its counter, which takes the line away from threads 1 to 3; thread 1 then writes its own counter and has to fetch the line back. The line ping-pongs between the cores on every increment, so each increment waits for a cache-to-cache transfer instead of hitting the core's own L1 cache. The threads share the hardware unit (the line) even though they do not share data, which is why it is called false sharing. With 4 threads, adjacent took 572.50 ms on average against 132.54 ms for padded.

2. Why does padding fix the problem, and what does it cost? Why was the effect absent or small with 1 thread?

Padding puts each counter 16 longs (128 bytes) away from the next, so every counter is on its own cache line (128 bytes also clears the adjacent-line prefetcher, which works on pairs of lines). Each core can then keep its own line in its cache in the modified state and update it without telling anyone, so there is no invalidation traffic and the threads run fully independently: padded stayed at about 129 to 141 ms from 1 to 8 threads, because each thread does the same work on its own core.

The cost is memory: here 128 bytes per counter instead of 8, i.e. 16 times more, and in a real structure the padding wastes cache capacity and makes the code less obvious.

With 1 thread there is no other core to steal the line, so the line stays in that core's cache and adjacent and padded are the same (130.06 ms against 128.84 ms, slowdown 1.01).

3. Which of the three versions is fastest? Explain why local accumulation is the best remedy in practice, and when padding is still the right tool.

local-accumulate was the fastest in every run (35 to 41 ms with 4 threads, against about 132 ms for padded and 572 ms for adjacent). It counts in a local variable, which the JIT keeps in a register, and writes the shared array only once at the end. So there is no shared memory traffic at all inside the loop, not even to the thread's own cache line, and the loop is as cheap as a loop can be.

That is why local accumulation is the best remedy in practice: it removes the sharing instead of working around it, and it is exactly how reductions such as lock-partial and PLINQ in Task 1 work. Padding is still the right tool when the per-thread state has to stay in shared memory while the threads run, for example counters or statistics that another thread reads while they are being updated, per-thread queue heads, or locks and flags that sit next to each other in an array or object.

4. How did the slowdown change as the number of threads grew, and what would you expect if the threads ran on a single core?

The slowdown of adjacent over padded grew with the number of threads: 1.01x with 1 thread, 2.21x with 2, 4.32x with 4 (average of 3 runs) and 5.94x with 8. More threads means more cores fighting over the same line, so each core waits longer before it gets the line back, and the adjacent time rose from 130 ms to 838 ms while padded stayed almost flat.

If all the threads ran on a single core, there would be only one cache, so the line would never move between cores and there would be no invalidations. The adjacent and padded versions would then run at the same speed; the threads would just take turns, as in the 1-thread case. False sharing only appears when threads run in parallel on different cores.

5. How would you recognize false sharing in a real program, where the counters are not so obvious? Name one symptom or tool.

Symptoms: a program that is correct but scales badly or even gets slower as threads are added, although the threads do not share any data logically; a large difference between runs with the threads on the same core and on different cores; and the speed changing when unrelated fields are moved around or when a structure is padded.

Tools: a hardware-counter profiler shows a large number of cache misses and cache-line transfers between cores. On Windows, Intel VTune (Memory Access analysis) or the CPU usage and hardware counters in Visual Studio's profiler; on Linux, perf c2c reports exactly which cache lines are shared between cores and which fields in them are written by different threads. In .NET a common fix, once found, is to keep the data local or to separate it, for example with [StructLayout(LayoutKind.Explicit)] and explicit padding.

Section 5: Task 4 - Scheduling Policy Summary

The uniform workload was run with: dotnet run --no-build -c Release -- 4 uniform. The uneven columns are copied from Task 2.

Summary of the two workloads (4 threads)

Policy	Uneven: wall (ms)	Uneven: imbalance	Uniform: wall (ms)	Uniform: imbalance	Verdict
static	132.27	6.96	111.86	1.17	Worst on uneven; middle on uniform
static,1	114.35	1.22	113.33	1.32	Better on uneven; slowest on uniform
static,64	106.09	1.05	108.00	1.38	Good on uneven; average on uniform
dynamic,1	98.57	1.00	105.96	1.00	Well balanced on both
dynamic,64	102.11	1.02	94.96	1.02	Good on both
guided,1	92.82	1.03	92.60	1.02	Good on both
guided,64	83.77	1.02	88.96	1.04	BEST on uneven; 2nd on uniform
Parallel.For	90.77	1.00	85.50	1.00	BEST on uniform; 2nd on uneven

```
C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab7_PDC\SchedulingBench>dotnet run --no-build -c Release -- 4 uniform
Workload: uniform  Threads: 4  Iterations: 10000  ProcessorCount: 8  (best of 3 runs)
Serial time: 299.39 ms
static        wall=   111.86 ms  busy_max=   111.23 ms  busy_min=    95.45 ms  imbalance=  1.17  correct=YES
static,1      wall=   113.33 ms  busy_max=   112.87 ms  busy_min=    85.81 ms  imbalance=  1.32  correct=YES
static,64     wall=   108.00 ms  busy_max=   107.52 ms  busy_min=    77.75 ms  imbalance=  1.38  correct=YES
dynamic,1     wall=   105.96 ms  busy_max=   105.58 ms  busy_min=   105.25 ms  imbalance=  1.00  correct=YES
dynamic,64    wall=    94.96 ms  busy_max=    94.66 ms  busy_min=    92.37 ms  imbalance=  1.02  correct=YES
guided,1      wall=    92.60 ms  busy_max=    92.35 ms  busy_min=    90.87 ms  imbalance=  1.02  correct=YES
guided,64     wall=    88.96 ms  busy_max=    88.73 ms  busy_min=    85.72 ms  imbalance=  1.04  correct=YES
Parallel.For  wall=    85.50 ms  busy_max=    85.45 ms  busy_min=    85.08 ms  imbalance=  1.00  correct=YES
```

Written summary

On the uneven workload, where the cost of iteration i grows with i, plain static was clearly the worst policy: it took 132.27 ms with an imbalance of 6.96, because the last thread received the most expensive quarter of the loop while the first thread finished early.

Every policy that interleaves or shares out the iterations fixed this, and guided,64 was the best on my machine at 83.77 ms (imbalance 1.02), followed by Parallel.For (90.77 ms) and guided,1 (92.82 ms).

Guided suits this workload because its first large chunks are taken from the cheap start of the loop, so there are few claims, while the small chunks at the expensive end let all threads finish at almost the same moment.

On the uniform workload every iteration costs the same, so all policies were much closer together (85.50 to 113.33 ms) and the best was Parallel.For at 85.50 ms, with guided,64 second at 88.96 ms.

Static was not the winner on the uniform workload either (111.86 ms, imbalance 1.17): even with identical work per thread the threads did not finish together, because the 8 logical cores do not all run at the same speed when threads share a physical core, and static cannot move work to the faster threads.

So the measurements show that balancing at run time (dynamic, guided or Parallel.For) is worth its small overhead whenever the cost per iteration or the speed of the cores is uneven, and costs almost nothing when it is not.

My rule of thumb is: use static only when the iterations are known to cost the same, are very cheap, and the machine is otherwise idle; otherwise start with Parallel.For or a guided/dynamic schedule with a moderate chunk size such as 64, and measure.

Discussion Questions

1. Which policy suited the uneven workload best on your machine, and which suited the uniform workload best? Support each answer with numbers from your table.

Uneven workload: guided,64 was the best at 83.77 ms with an imbalance of 1.02, followed by Parallel.For (90.77 ms, 1.00) and guided,1 (92.82 ms, 1.03). Plain static was the worst at 132.27 ms with an imbalance of 6.96, so guided,64 was about 1.6 times faster. Compared with the serial time of 310.25 ms that is a speedup of 3.70 on 4 threads, against 2.35 for static.

Uniform workload: Parallel.For was the best at 85.50 ms (imbalance 1.00), followed by guided,64 (88.96 ms) and guided,1 (92.60 ms). Static took 111.86 ms (imbalance 1.17) and static,1 was the slowest at 113.33 ms. Guided,64 was in the top two on both workloads.

2. Why can static be the best or one of the best choices for the uniform workload, even though it was the worst for the uneven one?

With a uniform workload every block contains the same amount of work, so static already gives each thread an equal share without any run-time coordination: no shared counter, no atomic operations, no contention, and each thread works on one contiguous block with good locality. When the threads run at the same speed, it is the cheapest possible schedule, and it can be the best choice.

On my machine it was not the best (111.86 ms against 85.50 ms for Parallel.For), and the reason is visible in its imbalance of 1.17: even with equal work, the threads did not finish together. The 8 logical processors are not all equally fast at a given moment, for example when two threads share a physical core through hyper-threading or the OS runs something else, and static cannot move work away from a slow thread. Static is only optimal when both the work and the processors are uniform.

3. If you did not know in advance how uneven the iterations are, which policy would you choose as a safe default, and why?

I would choose Parallel.For (or, when writing the schedule by hand, guided or dynamic with a moderate chunk size such as 64). In my measurements these were never far from the best on either workload: Parallel.For was 90.77 ms on uneven and the best on uniform, and guided,64 was the best on uneven and second on uniform.

A dynamic policy balances the work at run time, so it protects against uneven iterations and against threads that run slower, and the cost of that protection is small when the iterations are not tiny. Static is the risky choice: when the work is uneven it can be very bad (6.96 imbalance here), while the best it can gain on a uniform workload is a small amount of overhead.

4. What would change in your recommendation if each iteration were very cheap (a few nanoseconds), so that claiming a chunk costs as much as the work itself?

If each iteration costs only a few nanoseconds, claiming a chunk from a shared counter (an atomic operation on a contended cache line, typically tens of nanoseconds) would cost as much as or more than the work. dynamic,1 and guided,1 would then be much slower than static, because most of the time would go into claiming iterations rather than doing them.

I would then use static, which needs no claims at all, or dynamic and guided with large chunks (thousands of iterations), so that the claim cost is spread over a lot of work. Parallel.For also adapts its range sizes and is usually reasonable; for very cheap bodies Partitioner.Create(0, n, rangeSize) lets each task process a range in a tight inner loop. Task 1 showed the same effect: PLINQ's per-element overhead made it slower than a plain loop when the work per element was tiny.

5. Which of the hand-made policies resembles what Parallel.For does by itself, and what does that suggest about using the library default?

Parallel.For behaves most like guided (and dynamic): it does not split the loop into fixed blocks in advance. Its tasks take ranges of iterations from a shared source as they go, starting with larger ranges and handing out smaller ones as the work runs down, and threads that finish early take more work. That is why it showed an imbalance of 1.00 on both workloads and was close to the best hand-made policy in both cases.

This suggests using the library default first. It gave balanced, good performance on both workloads without choosing a policy or a chunk size, and it handles exceptions, cancellation and the thread pool for you. Hand-made schedules are worth it only when measurements show a specific need, such as very cheap iterations or a known cost pattern.

Section 6: GitHub Repository

All source code for this lab is pushed to the lab7 branch of my repository:

https://github.com/fahad1084/-PDC_Labs/tree/lab7